

Lab2 实验报告

实验目的

- 理解页表的建立和使用方法：** 掌握RISC-V Sv39三级页表结构，明确虚拟地址到物理地址的翻译逻辑，通过配置页表项、设置satp寄存器、刷新TLB等操作，实现内核虚拟地址映射，理解页表的"虚实地址翻译"核心作用。
- 理解物理内存的管理方法：** 学习从DTB提取物理内存范围，基于Page结构体对物理页建模，通过free_area维护空闲页链表，掌握物理内存从"硬件资源"到"软件可管理页"的转化过程，明确物理内存管理的基础逻辑。
- 理解页面分配算法：** 分析First-Fit算法的分配与释放逻辑，独立实现Best-Fit算法，对比两种连续分配策略的差异，理解内存碎片控制与分配效率的权衡，建立物理内存高效管理的思维。

实验内容

在Lab1实现可启动最小内核的基础上，本实验核心围绕操作系统的物理内存管理展开——为实现内存资源的高效使用，需完成三方面核心任务：

首先，了解系统物理内存的发现方式，明确内核可管理的物理内存范围；其次，学习建立物理内存的初步管理机制，重点掌握连续物理内存的管理逻辑；最后，掌握页表相关操作，通过建立页表实现虚拟内存到物理内存的映射，进而对段页式内存管理机制形成全面认知。

整体而言，实验需在Lab1基础上，完成物理内存管理模块的搭建，并建立最简单的页表映射，确保内核能通过页表实现虚实地址的正确翻译与内存访问。

实验过程

工具、文件准备

使用的工具与Lab1相同：RISC-V交叉编译工具链，RISC-V GDB调试器，QEMU模拟器（RISC-V版），Make工具，Git（代码管理），tmux（分屏调试）

实验代码从网站下载。

```
yuhaoxi@yuhaoxi:~/Desktop/lab2$ tree
```

```
├── kern
│   ├── debug
│   │   ├── assert.h
│   │   └── panic.c
│   ├── driver
│   │   ├── console.c
│   │   ├── console.h
│   │   ├── dtb.c
│   │   └── dtb.h
│   ├── init
│   │   ├── entry.S
│   │   └── init.c
│   ├── libs
│   │   └── stdio.c
│   └── mm
│       ├── best_fit_pmm.c
│       ├── best_fit_pmm.h
│       ├── default_pmm.c
│       ├── default_pmm.h
│       ├── memlayout.h
│       ├── mmu.h
│       ├── pmm.c
│       └── pmm.h
```

```
├── lab2.md
├── libs
│   ├── defs.h
│   ├── error.h
│   ├── list.h
│   ├── printfmt.c
│   ├── readline.c
│   ├── riscv.h
│   ├── sbi.c
│   ├── sbi.h
│   ├── stdarg.h
│   ├── stdio.h
│   ├── string.c
│   └── string.h
├── Makefile
└── tools
    ├── boot.ld
    ├── function.mk
    ├── gdbinit
    ├── grade.sh
    ├── kernel.ld
    └── kernel_nopage.ld
```

练习一

理解first-fit连续物理内存分配算法（思考题）

first-fit连续物理内存分配算法作为物理内存分配一个很基础的方法，需要同学们理解它的实现过程。请大家仔细阅读实验手册的教程并结合kern/mm/default_pmm.c中的相关代码，认真分析default_init, default_init_memmap, default_alloc_pages, default_free_pages等相关函数，并描述程序在进行物理内存分配的过程以及各个函数的作用。请在实验报告中简要说明你的设计实现过程。

请回答如下问题：

你的first fit算法是否有进一步的改进空间？

Ans： 实验提供的First-Fit算法通过free_area_t结构体管理物理内存，四个核心函数分工明确，形成初始化→内存探测→分配→释放的完整链路，接下来我们以给代码添加注释的方式来详细描述程序在进行物理内存分配的过程以及各个函数的作用。

1. default_init(void)

初始化空闲内存管理器

```
static void default_init(void) {
    list_init(&free_list); // 初始化空闲页双向链表（链表头指向自身）
    nr_free = 0;           // 初始化空闲页总数为0（此时无可用物理页）
}
```

调用时机： pmm_init() → init_pmm_manager()中调用，是物理内存管理的初始化第一步。

核心作用： 搭建空闲内存管理的基础框架，用free_list串起所有空闲页块，用nr_free记录总空闲页数，为后续内存探测和分配做准备。

2. default_init_memmap(struct Page *base, size_t n)

初始化物理内存页，构建空闲页块

```
static void default_init_memmap(struct Page *base, size_t n) {
    assert(n > 0);
    struct Page *p = base;

    // 1. 初始化每个物理页的基础状态
    for (; p != base + n; p++) {
        assert(PageReserved(p)); // 确保页初始为"内核保留"（未被使用）
        p->flags = p->property = 0; // 清空标志位和空闲块大小记录
        set_page_ref(p, 0);        // 引用计数设0（无进程使用此页）
    }

    // 2. 标记"空闲块首页"：记录块大小，设置"空闲块"标志
    base->property = n; // 首页的property字段记录整个空闲块的页数（n页）
    SetPageProperty(base); // 置位PG_property，标记此页是空闲块的首页

    // 3. 加入空闲链表，更新空闲页总数
    nr_free += n;
    if (list_empty(&free_list)) {
        list_add(&free_list, &(base->page_link)); // 链表空时直接添加
    } else {
        // 按物理地址升序插入（遍历链表找第一个比base大的页，插在前面）
        list_entry_t* le = &free_list;
        while ((le = list_next(le)) != &free_list) {
            struct Page* page = le2page(le, page_link);
            if (base < page) { // 地址比较：确保链表按地址从小到大排序
                list_add(&free_list, &(base->page_link));
            }
        }
    }
}
```

```

        list_add_before(1e, &(base->page_link));
        break;
    } else if (list_next(1e) == &free_list) { // 遍历到链表尾，直接追加
        list_add(1e, &(base->page_link));
    }
}
}
}

```

调用时机： kern_init() → pmm_init() → page_init() → init_memmap(), 是物理内存探测后的关键步骤。

核心作用： 将探测到的可用物理内存（如QEMU中的0x80000000~0x88000000）转化为软件可管理的空闲页块：

- 给每个物理页初始化flags（状态）、ref（引用计数）等字段；
- 标记空闲块的"首页"（用property记录块大小，PG_property标志标识）；
- 按物理地址升序插入free_list，保证后续分配时能按地址顺序遍历。

3. default_alloc_pages(size_t n)

执行First-Fit分配，返回空闲页首地址

```

static struct Page *default_alloc_pages(size_t n) {
    assert(n > 0);
    if (n > nr_free) { // 空闲页总数不足，分配失败
        return NULL;
    }
    struct Page *page = NULL;
    list_entry_t *1e = &free_list;

    // 1. 遍历空闲链表，找"第一个大小≥n"的空闲块（First-Fit核心逻辑）
    while ((1e = list_next(1e)) != &free_list) {
        struct Page *p = 1e2page(1e, page_link); // 链表节点转Page结构体
        if (p->property >= n) { // 找到足够大的块
            page = p;
            break;
        }
    }

    // 2. 若找到空闲块，处理分配逻辑
    if (page != NULL) {
        list_entry_t* prev = list_prev(&(page->page_link)); // 记录块的前驱节点
        list_del(&(page->page_link)); // 将该块从空闲链表中移除

        // 2.1 若块大小>n，分割块：剩余部分重新加入空闲链表
        if (page->property > n) {
            struct Page *p = page + n; // 剩余块的首页（跳过已分配的n页）
            p->property = page->property - n; // 剩余块大小 = 原大小 - 分配大小
            SetPageProperty(p); // 标记剩余块为空闲块首页
            list_add(prev, &(p->page_link)); // 插入到原块的前驱节点后
        }

        // 2.2 更新空闲页总数，标记已分配页的状态
        nr_free -= n;
        ClearPageProperty(page); // 清除"空闲块首页"标志（该页已分配）
    }
}

```

```

    }
    return page; // 返回已分配块的首页（NULL表示分配失败）
}

```

调用时机： 内核需要物理页时调用。

核心作用： 按"First-Fit"策略分配连续物理页：

- 从free_list表头开始遍历，优先选择"第一个足够大"的空闲块；
- 若块大小超过需求，分割后将剩余部分放回空闲链表；
- 移除已分配块的空闲标记，更新空闲页总数，确保后续分配逻辑正确。

4. default_free_pages(struct Page *base, size_t n)

释放物理页，合并相邻空闲块

```

static void default_free_pages(struct Page *base, size_t n) {
    assert(n > 0);
    struct Page *p = base;

    // 1. 初始化释放页的状态（标记为未使用）
    for (; p != base + n; p++) {
        assert(!PageReserved(p) && !PageProperty(p)); // 确保页非保留、非空闲
        p->flags = 0; // 清空状态标志
        set_page_ref(p, 0); // 引用计数设0（无进程引用）
    }

    // 2. 标记释放块为空闲块首页，更新空闲页总数
    base->property = n;
    SetPageProperty(base);
    nr_free += n;

    // 3. 按物理地址升序插入空闲链表
    if (list_empty(&free_list)) {
        list_add(&free_list, &(base->page_link));
    } else {
        list_entry_t* le = &free_list;
        while ((le = list_next(le)) != &free_list) {
            struct Page* page = le2page(le, page_link);
            if (base < page) {
                list_add_before(le, &(base->page_link));
                break;
            } else if (list_next(le) == &free_list) {
                list_add(le, &(base->page_link));
            }
        }
    }

    // 4. 合并"前驱空闲块"（若释放块的前一页是空闲块首页）
    list_entry_t* le = list_prev(&(base->page_link));
    if (le != &free_list) {
        p = le2page(le, page_link);
        if (p + p->property == base) { // 前驱块的末尾 == 释放块的开头
            p->property += base->property; // 合并大小
            ClearPageProperty(base); // 释放块不再是首页
            list_del(&(base->page_link)); // 从链表移除释放块
            base = p; // 合并后新块的首页是前驱块首页
        }
    }
}

```

```

    }
}

// 5. 合并"后继空闲块"（若释放块的后一页是空闲块首页）
le = list_next(&(base->page_link));
if (le != &free_list) {
    p = le2page(le, page_link);
    if (base + base->property == p) { // 释放块的末尾 == 后继块的开头
        base->property += p->property; // 合并大小
        clearPageProperty(p); // 后继块不再是首页
        list_del(&(p->page_link)); // 从链表移除后继块
    }
}
}
}

```

调用时机： 内核释放物理页时调用（如进程退出、页表销毁）。

核心作用： 安全释放物理页，并合并相邻空闲块，解决外部碎片问题：

- 初始化释放页的状态（清空标志、引用计数），标记为空闲块；
- 按地址升序插入空闲链表，保证链表有序性；

关键逻辑： 合并前驱和后继空闲块，若释放块与前后空闲块地址连续，合并为一个大空闲块，避免小块分散导致的外部碎片。

其他辅助函数

此外，代码中除了这四个核心函数，还定义了一些其他函数和结构体，这些函数和结构体主要是辅助统计、算法校验、管理器注册三类功能，作用简单清晰，具体如下：

default_nr_free_pages(void)： 对外提供当前空闲物理页总数的查询接口，比如内核想判断是否有足够空闲页分配时，会调用这个函数获取nr_free。

basic_check(void)： 对First-Fit算法做"最小化正确性校验"，覆盖分配不重复、空闲时分配失败、释放后可重新分配等基础场景，确保算法不会出现低级错误（如页地址重复、空闲数统计错误）。

default_check(void)： 对First-Fit算法做"全面正确性验证"，除了基础场景，还覆盖多页分配、块分割、碎片处理、块合并等核心场景，确保算法在复杂情况下也能正常工作（比如释放中间页后能合并前后空闲块）。

default_pmm_manager结构体： 是ucore内核的物理内存管理器注册接口，把First-Fit算法的所有函数打包成一个结构体，内核通过pmm_manager指针调用对应函数。

First-Fit物理内存分配完整流程

结合以上对各函数的分析，我们可以梳理First-Fit物理内存分配完整流程如下：

1. 初始化阶段

内核启动后调用pmm_init()，通过default_init()初始化空闲页双向链表free_list和空闲页总数nr_free；

随后解析DTB得到物理内存范围，调用default_init_memmap()将非内核占用的物理内存转化为空闲页块，加入free_list。

2. 分配阶段

当内核需要若干连续物理页时，调用default_alloc_pages(n)（n为所需页数），遍历free_list找第一个大小≥n的空闲块；

若找到，若块大小大于n，则分割为n页（分配）和剩余页（重新加入free_list），返回分配块的首页地址，内核基于此使用物理内存。

3. 释放阶段

内核释放物理页时，调用default_free_pages(base, n)（base为释放页起始页，n为页数），将释放页标记为空闲并插入free_list；

检查前后是否有空闲块，且是否与当前块相邻，若有则合并，最后更新nr_free，完成释放。

思考：这个First-Fit算法是否有改进空间？

结合以上真实的去梳理算法的流程，以及课堂上的讲解，不难看出First-Fit算法的核心优势是分配时遍历成本低，因为它找到第一个满足条件的块即停止，但也存在明显缺陷，下面探讨一些可能的改进方向：

1. 解决外部碎片问题：按块大小排序，减少碎片积累

问题： First-Fit按地址顺序遍历，分配后分割的小块可能分散在链表中，长期运行会产生大量外部碎片，总空闲页足够，但无连续块满足需求。

改进方法： 将空闲链表按块大小升序/降序排序，而非按地址排序，这也是Best-Fit算法/Worst-Fit算法的思想。

2. 提升分配效率：引入块大小索引，避免全链表遍历

问题： 当空闲块数量较多时，First-Fit需遍历整个链表找块，分配效率随块数量增加而下降。

改进方法： 建立块大小索引表，如按2的幂次分组：1页、2页、4页、8页...：

- 每个索引对应一个空闲链表，存储该大小范围的空闲块；
- 分配时直接定位到最小满足需求的索引，遍历对应链表，如分配5页，定位到8页索引的链表。

3. 优化内存利用率：支持非连续页分配，适配非连续需求

问题： First-Fit仅支持连续物理页分配，但部分场景无需连续物理页，连续分配会浪费空闲页。

改进方法： 结合页表机制，支持非连续物理页分配：

分配时无需找连续块，可将多个离散的物理页通过页表映射到连续的虚拟地址；典型实现如"伙伴系统"（按2的幂次分配，支持快速合并）或"Slub算法"（两级分配：先分配页，再在页内分配小内存单元），兼顾效率与利用率。

练习二

实现Best-Fit连续物理内存分配算法（需要编程）

在完成练习一后，参考kern/mm/default_pmm.c对First Fit算法的实现，编程实现Best Fit页面分配算法，算法的时空复杂度不做要求，能通过测试即可。请在实验报告中简要说明你的设计实现过程，阐述代码是如何对物理内存进行分配和释放，并回答如下问题：

你的Best-Fit算法是否有进一步的改进空间？

Ans： First-Fit和Best-Fit算法的核心差异集中在如何选择空闲块，因此其实只有alloc_pages函数需要修改，其余函数完全可以复用，我们在这里列出我们编写的Best-Fit算法的alloc_pages函数：

```
static struct Page *  
best_fit_alloc_pages(size_t n) {
```

```

assert(n > 0);
if (n > nr_free) {
    return NULL;
}
struct Page *page = NULL;
list_entry_t *le = &free_list;
size_t min_size = nr_free + 1; // 初始化为比最大可能值大1的数

/*LAB2 EXERCISE 2: 2312642*/
// 下面的代码是first-fit的部分代码，请修改下面的代码改为best-fit
// 遍历空闲链表，查找满足需求的空闲页框
// 如果找到满足需求的页面，记录该页面以及当前找到的最小连续空闲页框数量
while ((le = list_next(le)) != &free_list) {
    struct Page *p = le2page(le, page_link);
    // 找到大小足够且更接近需求的块
    if (p->property >= n && p->property < min_size) {
        min_size = p->property; // 更新最小块大小
        page = p; // 记录当前最优块
    }
}

if (page != NULL) {
    list_entry_t* prev = list_prev(&(page->page_link));
    list_del(&(page->page_link));
    if (page->property > n) {
        struct Page *p = page + n;
        p->property = page->property - n;
        setPageProperty(p);
        list_add(prev, &(p->page_link));
    }
    nr_free -= n;
    clearPageProperty(page);
}
return page;
}

```

那么代码对物理内存进行分配和释放的流程，变化主要也就体现在选择空闲块这一步上，因此再次阐述此函数的设计思路与核心逻辑：

1. 前置校验

首先检查请求的页数 n 是否大于总空闲页数 nr_free ，若大于则直接返回NULL。

定义 min_size 变量，用于记录当前找到的最小满足需求的块大小；定义 $page$ 指针用于记录最优块。

2. 遍历空闲链表，筛选最优块

遍历整个空闲链表，对每个空闲块：

- 若块大小 $\geq n$ ，且块大小 $< min_size$ （比当前记录的最优块更接近需求），则更新 min_size 为当前块大小，并记录当前块为 $page$ 。

遍历完成后， $page$ 指向的就是大小最接近需求的块。

3. 处理选中的块

若找到最优块，先将该块从空闲链表中移除，避免重复分配。

若块大小 $> n$ （块有剩余空间），则分割块：

- 从块的第n页开始作为新的空闲块，设置其大小为原大小 - n，并标记为空闲块。

将新块插入到原块的前驱节点后，维持链表有序性。

4. 更新状态并返回

总空闲页数减去已分配的n页。

清除已分配块的空闲标记，返回该块的首地址。

与First-Fit的关键区别：

- First-Fit找到第一个满足需求的块就停止遍历，而Best-Fit会遍历所有块，选择剩余空间最小的块。这种策略使Best-Fit能更好地保留大空闲块，减少外部碎片，但代价是遍历成本更高。

代码验证

接下来我们来进行代码的验证：

首先我们执行make和make qemu命令，编译代码并运行内核：

```
yuhaoxi@yuhaoxi:~/Desktop/lab2$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/debug/panic.c
+ cc kern/driver/console.c
+ cc kern/driver/dtb.c
+ cc kern/mm/best_fit_pmm.c
+ cc kern/mm/default_pmm.c
+ cc kern/mm/pmm.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
yuhaoxi@yuhaoxi:~/Desktop/lab2$ make qemu
```

```
OpenSBI v0.4 (Jul  2 2019 11:53:53)
```

```

      _ _ _ _ _      _ _ _ _ _ _ _ _ _ _
     / _ \          / _ \| _ \| _ \|
    | | | | _ _ _ _ _ | (___| |_) | | | | |
    | | | | ' _ \| _ \| _ \| _ \| _ \|
    | |_| | |_) | _ \| | | |_) | | |
    \___/| . _/ \___|_| | |___/|___/|___|
        | |
        |_|
```

```

Ubuntu Software      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs    : 8
Current Hart         : 0
Firmware Base         : 0x80000000
Firmware Size         : 112 KB
Runtime SBI Version   : 0.1
```

```

PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xfffffffffffff (A,R,W,X)
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000080000000 (128 MB)
  End: 0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...
```

```

Special kernel symbols:
  entry 0xffffffffc02000d8 (virtual)
  etext 0xffffffffc020164a (virtual)
  edata 0xffffffffc0205018 (virtual)
  end    0xffffffffc0205078 (virtual)
Kernel executable memory footprint: 20KB
memory management: best_fit_pmm_manager
physical memory map:
  memory: 0x0000000080000000, [0x0000000080000000, 0x0000000087ffffff].
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0204000
satp physical address: 0x0000000080204000
```

从输出结果来看，Best-Fit算法实现是成功的，关键观察如下：

算法切换成功：日志中明确显示memory management: best_fit_pmm_manager，说明系统已成功切换到Best-Fit内存管理器，而非默认的First-Fit。

测试通过：check_alloc_page() succeeded! 表示best_fit_check()函数（自动测试用例）执行完毕且未触发任何断言失败，说明所有测试点都通过了验证。

物理内存映射正常：physical memory map输出了正确的内存范围（Base: 0x80000000，Size: 128 MB），说明内存探测和初始化逻辑正常。

另外实验还提供了一个自动评分脚本，我们也来执行一下：

同样通过测试。

Ans: Best-Fit算法的核心思想是，在所有能满足分配请求的空闲块中，选择大小最接近请求大小的块进行分配，一定程度上解决了碎片问题，但仍存在缺陷，下面是一些可能的改进方向：

- ## 扩展练习Challenge1

Buddy System算法把系统中的可用存储空间划分为存储块(Block)来进行管理, 每个存储块的大小必须是2的n次幂(Pow(2, n)), 即1, 2, 4, 8, 16, 32, 64, 128...

Ans: 设计文档请见单独的"Buddy system设计文档", 已上传至同目录下。

扩展练习Challenge2

slub算法，实现两层架构的高效内存单元分配，第一层是基于页大小的内存分配，第二层是在第一层基础上实现基于任意大小的内存分配。可简化实现，能够体现其主体思想即可。

参考linux的slub分配算法，在ucore中实现slub分配算法。要求有比较充分的测试用例说明实现的正确性，需要有设计文档。

Ans: 设计文档请见单独的"slub设计文档", 已上传至同目录下。

扩展练习Challenge3

硬件的可用物理内存范围的获取方法（思考题）

如果OS无法提前知道当前硬件的可用物理内存范围，请问你有什么办法让OS获取可用物理内存范围？

Ans: 利用BIOS/UEFI接口查询内存映射

思路: 在PC兼容机上，BIOS（Basic Input/Output System）或UEFI（Unified Extensible Firmware Interface）在启动时会扫描硬件并构建内存映射表。OS可以调用这些固件提供的接口来获取信息，而无需自己探测。这是最可靠的方式，因为它直接来自硬件固件，避免了潜在的冲突或错误。

具体步骤:

- 在引导加载器阶段，使用BIOS中断调用。例如，在x86实模式下，通过INT 15h（功能码E820h）查询内存映射：
 1. 调用前设置寄存器：AX = E820h, EBX = 0（起始续接点），ES:DI指向缓冲区。
 2. BIOS返回一个结构数组，每个结构描述一个内存区域（起始地址、长度、类型，如可用内存、保留区、ACPI表等）。
 3. 如果EBX不为0，继续调用直到遍历完所有区域。
- 对于UEFI系统，使用EFI_BOOT_SERVICES.GetMemoryMap()函数调用，获取一个EFI_MEMORY_DESCRIPTOR数组，包含类似信息。
- Bootloader将这些信息打包传递给OS内核，内核在初始化内存管理器时使用。

讨论

实验重要知识点与OS原理知识点的关联分析

实验中的重要知识点	对应的OS原理知识点	含义、关系与差异理解
页表的建立与虚拟地址的映射	虚拟内存管理与地址空间隔离机制	含义： 通过建立RISC-V Sv39三级页表，设置satp寄存器并刷新TLB，实现虚拟地址到物理地址的正确映射。 关系： 实验层面通过具体寄存器和页表项操作，验证了虚拟地址空间的建立过程；理论上，它体现了操作系统为每个进程提供独立、受保护地址空间的抽象机制。 差异： 实验实现的是简化的单内核页表映射，而操作系统原理中强调多级页表在多进程环境下的动态切换与保护机制。
First-Fit连续内存分配算法	连续内存分配策略与外部碎片问题	含义： First-Fit算法是按地址顺序遍历空闲链表，直到找到第一个足够大的块进行分配。 关系： 实验展示了内存分配与释放的完整流程：从查找空闲块、分割大块到释放合并。它反映了OS内存管理在性能与碎片控制间的基本权衡。 差异： First-Fit算法实现简单但易产生碎片；理论上，OS会结合Best-Fit、Buddy等策略进行优化，以兼顾分配速度与空间利用率。
Best-Fit连续内存分配算法	内存分配策略优化与碎片控制机制	含义： Best-Fit算法是通过遍历所有空闲块，选择最接近需求大小的块，从而优化外部碎片问题。 关系： Best-Fit能更好地利用内存空间，体现了"最优适配"的思想。它与First-Fit的对比体现了不同分配策略下系统性能的取舍。 差异： 实现的时间复杂度较高，而现代操作系统通常采用多级空闲链表或分组索引（如伙伴系统）来降低遍历成本。
物理页的释放与合并	外部碎片回收与内存再利用机制	含义： 在释放物理页时检查前后是否为相邻空闲块，若连续则进行合并。 关系： 合并机制是内存分配算法的重要补充，确保长期运行中不会因为小碎片导致"总空闲足够但无法分配大块"的问题。 差异： 实验采用线性检测与合并，而OS原理中的伙伴系统通过位运算快速定位并合并"伙伴块"，效率更高。
TLB刷新与上下文切换	地址转换缓存（TLB）与进程上下文切换机制	含义： 修改页表后，通过刷新TLB保证新映射生效。 关系： 实验操作TLB体现了硬件缓存与OS地址空间管理的协同关系，是虚拟内存系统性能优化的重要部分。 差异： 实验只涉及单进程内核态，而OS原理中还需考虑多进程切换时的TLB失效与ASID（地址空间标识符）管理。

OS原理重要但实验未覆盖的知识点

1. 用户态和内核态的保护机制： 操作系统通过页表项中的权限位以及特权级机制，实现对用户进程与内核空间的访问隔离。这是操作系统内核安全的基础。通过区分用户态与内核态，系统能防止进程非法访问内核或其他进程的内存区域。实验中仅构建了单一内核空间映射，未引入用户进程与系统调用机制，所以未涉及内存保护与权限控制逻辑。

2. 多进程与地址空间隔离： 每个进程拥有独立的虚拟地址空间，OS通过多级页表与上下文切换机制实现进程间内存隔离。地址空间隔离保证了进程的独立性与安全性，是实现多任务操作系统的核心。本实验运行在单内核模式下，未引入用户态进程、进程控制块与上下文切换，所以没有涉及多进程地址空间管理。

3. 反向映射：反向映射是一种通过物理页反查所有引用它的页表项（PTE）的机制，用于建立从物理地址到虚拟地址的逆向映射关系。反向映射是虚拟内存管理中的关键机制。它使得系统在执行页回收或换页时，能够快速找到并解除所有对某个物理页的引用，从而安全释放该页。本实验仅实现了从虚拟地址到物理地址的正向映射，未涉及从物理页反查页表项的反向映射。

实验总结和心得体会

实验总结

本次Lab2围绕ucore操作系统的物理内存管理机制展开，重点完成了两个核心练习：

First-Fit算法分析与理解：通过深入剖析default_pmm.c中的default_init、default_init_memmap、default_alloc_pages、default_free_pages等关键函数，系统掌握了First-Fit连续物理内存分配算法的实现原理。从空闲链表初始化、内存映射建立，到页面分配时的首次适配查找与释放时的相邻块合并，完整梳理了物理内存管理的全流程，验证了链表数据结构在内存管理中的核心作用。

Best-Fit算法实现与优化：在理解First-Fit基础上，独立编程实现了Best-Fit页面分配算法。通过遍历空闲链表寻找大小最匹配的内存块，优化内存利用率；在释放时维护地址有序链表并执行前后向合并，有效减少外部碎片。算法通过测试验证，展现了不同分配策略在时空效率与碎片控制方面的权衡。

并且，完成了Buddy System伙伴系统分配算法和SLUB分配算法的实现与验证的扩展练习：

Buddy System分配算法实现：通过分析伙伴系统的核心原理，设计了基于完全二叉树结构的内存块管理机制，实现了内存块的按2的n次幂划分、分配时的分裂机制和释放时的伙伴合并算法。验证了位图索引在快速查找伙伴块中的关键作用，确保了外部碎片的最小化。

SLUB分配算法实现：构建了两层架构的高效内存分配系统，第一层基于页分配器获取连续物理页框，第二层在页内实现任意大小对象缓存管理。通过建立kmalloc_caches数组分类管理不同大小的对象，验证了SLAB分配器在减少内部碎片和提升分配效率方面的优势。

实验最终达成目标，既掌握了物理内存分配算法的核心原理与实现细节，也通过对比不同算法的特性深化了对内存碎片问题的理解，同时提升了在操作系统内核层面进行内存管理模块编程的实践能力，为后续虚拟内存管理的学习打下了坚实基础。

心得体会

通过本次实验，我们对操作系统的内存管理机制有了更深入的理解。课堂上学习的“页表”“虚拟内存”“物理页分配”等概念，在实验中得到了直观的体现。

在实验中，通过分析first-fit算法，编写best-fit算法、Buddy System伙伴系统分配算法和SLUB分配算法，让我们体会到不同算法在空间利用率、执行效率和碎片控制上的权衡。让我们从代码层面理解了这些算法的实现原理和取舍策略。

也让我们认识到数据结构在设计中的核心作用。无论是free_list双向链表，还是Page结构体中的property和flags字段，它们共同构建了内存管理的数据系统，使得软件能够高效地管理和调度硬件资源。

最后，系统级编程要求严谨的细节控制。一个未被刷新的TLB、一个未正确初始化的链表节点，都可能导致系统崩溃。这要求我们不仅要理解宏观架构，也要掌握底层机制，具备从指令集、硬件行为到软件逻辑的全栈调试能力。

本次实验不仅让我们掌握了内存管理的核心技术，也锻炼了我们从原理到实现、从调试到优化的完整工程能力，为后续学习进程管理、文件系统等模块奠定了坚实基础。